

TRIGUNAI INNOVATIONS



Acharya

THE GUIDE · आचार्य

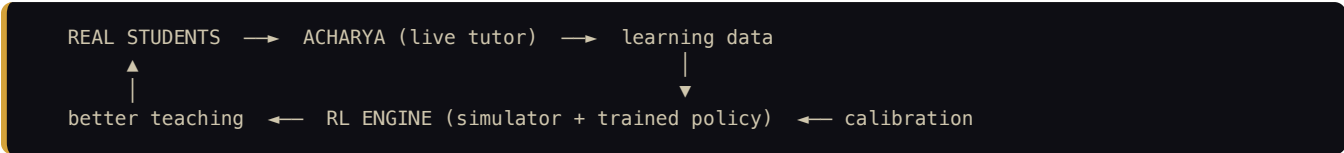
The TrigunAI Tutor Pipeline

How a live AI tutor and a reinforcement-learning engine
teach — and keep getting better. The whole system,
explained.

Internal & Confidential · 2026-06-29

o. TL;DR (read this first)

We run a live, one-on-one AI tutor called **Acharya** that teaches people to build AI (over WhatsApp + web). Separately, we're building a **reinforcement-learning system that learns *how to teach better*** by practising against a simulated student. A privacy-safe **data loop** feeds what real students do back into that simulator, so the teaching keeps improving. Today the live tutor works and is pedagogically strong; the RL engine works end-to-end as research; the data loop is plumbed and just started collecting.



That circle is the whole product thesis: **teaching that measurably gets better over time.**

1. The question that started it

"Is there a scientifically-proven best teaching system — not just information transfer — and can we train one?"

The honest answer:

1. **There is no magic pre-trained "teacher model."** Teaching quality comes from *method* (pedagogy) layered on a capable language model — not from special weights.
2. **The method that works is well-established by learning science:** explain one idea at a time, make the learner *recall* before you tell them, give immediate specific feedback, space the review out over days, and actively *repair wrong mental models*. Done 1-on-1, this is worth roughly **+2 standard deviations** over a classroom lecture (Bloom's "2-sigma").
3. **The extra margin on top** — *which* concept/question/hint to give *next* — is a sequencing decision you can **learn with reinforcement learning**.

So we built two things: a **live tutor that already uses the proven method**, and an **RL engine that learns the optimal sequencing** and feeds improvements back. The rest of this doc explains both and how they connect.

2. The two halves of the system

What it is	Acharya — a real AI tutor students talk to	An RL system that learns to teach, in simulation
Where it runs	The Gurukul VM (20.219.2.53)	<code>tutor_rl/</code> in this repo (laptop/GPU)
Who touches it	Real students, over WhatsApp + web	Us, offline
Status	Live, in daily use	Works end-to-end as research
Risk model	Never disrupt students	Free to experiment

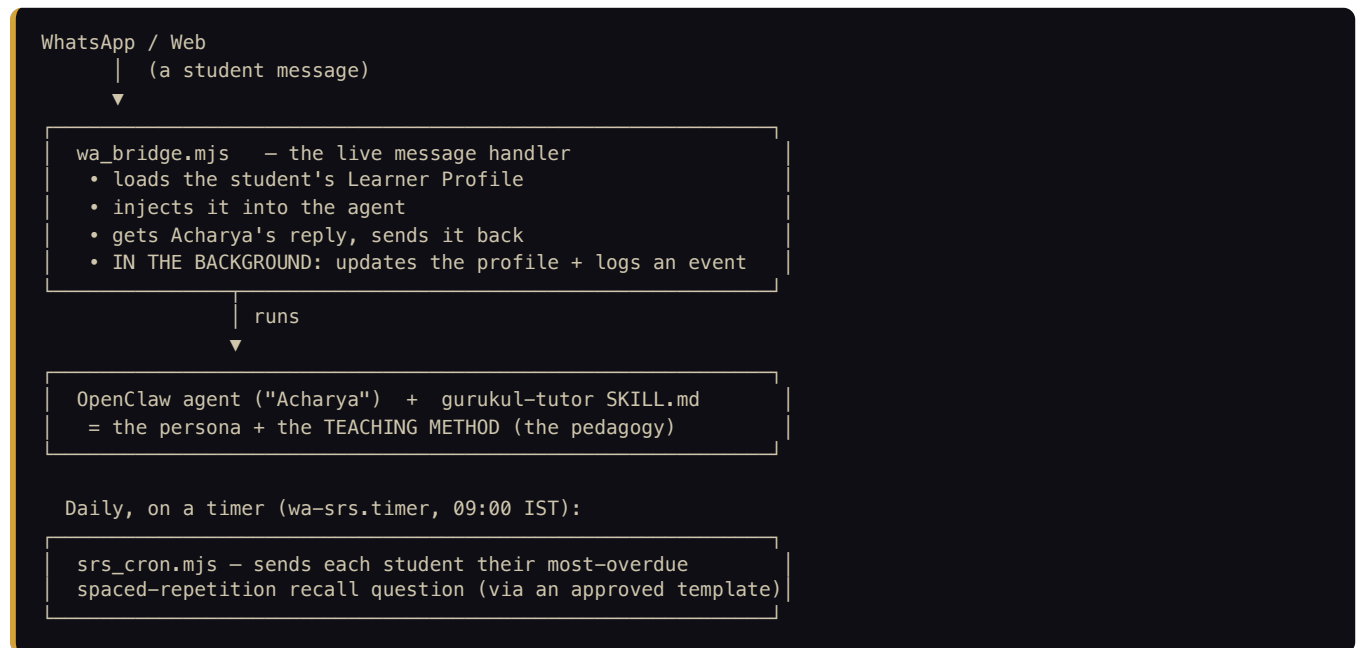
They are deliberately separate. The research engine can run wild without ever touching a real student; only *vett*ed improvements cross over into the live tutor.

3. Half A — the live tutor (Acharya)

3.1 What a student experiences

A student messages a WhatsApp number (or uses the web chat). Acharya, a warm "guru" persona, teaches them to build AI agents **one concept at a time, on their own project**, in short back-and-forth turns. Every day it also sends a quick "recall" question to refresh something they learned earlier.

3.2 The pieces (all on the VM)



3.3 The teaching method (the important part)

The method lives in `gurukul-tutor/SKILL.md`. It already encodes the proven principles:

- **One concept at a time**, taught in a strict prerequisite order (no skipping).
- **Mastery gate**: don't advance until the student can answer a check question unaided.
- **Spaced repetition (SRS)**: each learned concept comes back for review at expanding intervals ($1 \rightarrow 3 \rightarrow 7 \rightarrow 16 \rightarrow 30$ days), via the daily ping.
- **A Learner Model** per student: what they know, what they're shaky on, their misconceptions, their streak, their personal interests (used to tailor examples).

On 2026-06-29 we added the two highest-value upgrades the research pointed to: - **Elicit-before-explain** — make the student *guess/predict first*, then reveal. A real attempt before being told makes the idea stick far better. - **Misconception repair** — when a wrong answer reveals a *specific wrong model*, don't just re-explain; **name it, break it with a counterexample, then re-check**.

3.4 Who "owns" what (a key safety design)

The chatty AI proposes soft updates ("this concept was discussed", "new misconception spotted"), but **the code owns mastery and scheduling** — only a *deterministically graded correct recall* promotes a concept to "solid" and advances its review schedule. This stops the language model from rubber-stamping progress it didn't verify.

3.5 The WhatsApp channel (made fully live 2026-06-29)

Proactive messages (the daily recall, broadcasts) require **Meta-approved templates**. Now in place: - **Production WABA** (WhatsApp Business Account) id `1017321330664208` ("TrigunAI Innovations"). - Templates `gurukul_recall` (daily SRS ping) and `gurukul_announce` (broadcasts), both **APPROVED**, English (US). - `wa-srs.timer` fires daily at

4. Half B — the research engine (`tutor_rl/`)

The goal: **learn the optimal teaching sequence** instead of hand-writing it. This is a reinforcement-learning problem, and it's the same shape as TrigunAI's drone work (train a policy in a simulator, transfer to reality).

4.1 The core analogy (why we're good at this)

Drone RL (we already do this)	Tutor RL (this engine)
Physics simulator	Student simulator
Drone control policy	Tutor policy (the agent)
Sensor readings (partial view)	Student's answers (we can't see their true mind)
Motor command	Teaching move (ask / explain / hint / practise / review)
Reward = reached the goal	Reward = the student actually learned (and retained) it
Real-world flight test	Real students — used only to <i>judge</i> , never to train on

4.2 The student simulator (the heart of it)

To train a tutor you need something to teach. We can't train against thousands of real humans, so we **simulate a student** — exactly like simulating physics for the drone.

Each simulated student has a hidden inner state: how much they know of each concept, which **misconceptions** they hold, their mood (confidence/frustration/engagement), and personal traits (learning speed, forgetting rate). The tutor **can't see this directly** — it only sees the student's answers and has to *infer* what they know (just like drone sensors vs. true physics).

Every teaching move updates that hidden state using **learning-science rules**: - a good recall attempt at the right difficulty → durable learning; - just *telling* the answer → tiny gain, and it *reinforces* misconceptions; - spacing + practice → the concept *consolidates* (forgets slower); - too-hard repeatedly → frustration → the student disengages.

The **reward** is measured on a *held-out quiz* (questions not used in teaching, so the tutor can't "teach to the test"), plus a **retention re-test after simulated forgetting** — so cramming loses and real teaching wins.

4.3 The tutor (the thing being trained)

A policy that, given what it can observe, picks the next move: which concept, ask vs. tell, hint level, when to review, when to stop. We train it with a simple RL algorithm (REINFORCE) against thousands of *randomly varied* simulated students (so it generalizes, not memorizes).

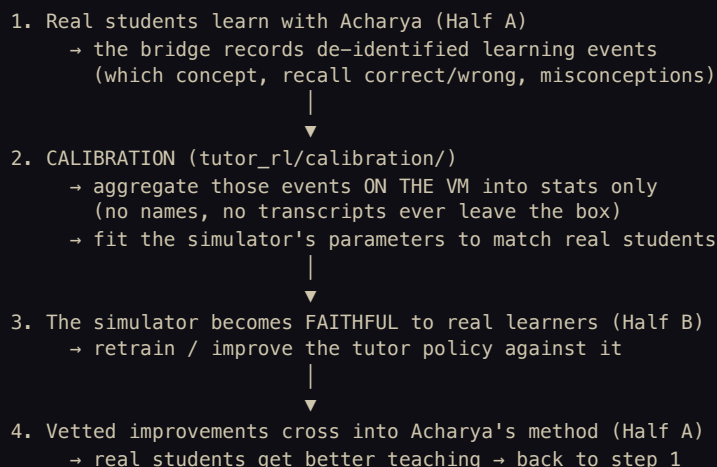
4.4 What the research run found (honest)

Against 300 unseen simulated students, the RL tutor: - **learned real pedagogy from scratch** — it *never* answer-dumps, and it **repairs more misconceptions than any other tutor**; - **beats** random / answer-dumping / fixed-order teaching; - but **does not yet beat a strong hand-built Socratic tutor** on retained learning (−16%) — expected for a tiny linear model, and we deliberately didn't over-tune an uncalibrated sim.

The most useful output wasn't the score — it was the lesson: the engine told us *which* levers matter most (no answer-dumping; active misconception repair). We then applied exactly those to the live Acharya prompt (§3.3). That is the research engine paying off immediately.

5. The loop that connects them (the flywheel)

This is what makes the whole thing compound instead of being two disconnected projects.



The golden rule: real students are the **judge, never the training set** — the "eye outside the optimizer." If the simulator's score improves but real-student learning doesn't, the *simulator* is wrong and we fix it. (Same discipline as the drone pipeline's independent visual critic.)

Privacy by design

Calibration runs **on the VM** and emits **only counts and distributions** — concept-state tallies, misconception frequencies, recall-success rates. No student names, phone numbers, or message text are ever copied off the box.

Current state of the loop (honest)

- The capture mechanism is **well-built and now active** (event logging went live 2026-06-29).
- First pull: **33 real misconceptions captured** from 7 students — richer and different from our hand-written guesses (proving the loop's value).
- **Still too thin** to fully calibrate (small cohort; the daily recall loop only just got unblocked). The plumbing is done; the data sharpens as the cohort grows and answers the daily recalls (which now actually send).

6. Follow one student (the whole system in a story)

1. **Priya** messages the WhatsApp number. `wa_bridge` loads her (empty) profile, Acharya welcomes her and asks *why* she wants to learn AI agents.
2. Acharya starts at concept #1. Using **elicit-before-explain**, it asks her to *guess* what makes something an "agent" vs. a chatbot. She guesses "it talks back."
3. That's a **known misconception** ("agent = chatbot"). Acharya **repairs** it: "that's the common trap — then how does a chatbot *book your flight*?" — a counterexample. Then it explains the real idea and asks a check question. She gets it → concept marked progressing.
4. In the background, `wa_bridge` updates her Learner Profile and writes a **learning event** (`concept discussed` , `misconception repaired`).
5. Three days later, `wa-srs.timer` fires; `srs_cron` sends her a `gurukul_recall` ping: "Quick recall: agent vs chatbot?" She answers correctly → the code grades it → the concept becomes **"solid"** and its next review is pushed further out (spaced repetition).
6. Every week, **calibration** aggregates everyone's events (de-identified) → the **simulator** learns that "agent = chatbot" is a common, sticky misconception → we **retrain the policy** and, if it confirms a better tactic, **update**

Acharya's method → Priya's cohort-mates get a better lesson. The circle closes.

7. Component reference (where everything lives)

Live tutor (on the VM `dk_trigun@20.219.2.53`)

Thing	Path / id
Message handler	<code>~/wa_bridge.mjs</code> (systemd <code>wa-bridge</code>)
Teaching method	<code>~/openclaw/workspace/skills/gurukul-tutor/SKILL.md</code>
Daily recall engine	<code>~/openclaw/gurukul/srs_cron.mjs</code> (systemd <code>wa-srs.timer</code> , 09:00 IST)
Student profiles	<code>~/openclaw/students/<wa_id>.json</code>
Learning-event stream	<code>~/openclaw/gurukul/events.jsonl</code>
Course concept banks	<code>~/openclaw/gurukul/courses/<course>.json</code>
WhatsApp config	<code>~/openclaw/wa_cloud.env</code> (WABA <code>1017321330664208</code>)

Research engine (this repo, `tutor_rl/`)

Thing	Path
Design spec	<code>TUTOR_RL_STUDENT_SIMULATOR_SPEC.md</code> (repo root)
Skills + misconception bank	<code>tutor_rl/knowledge.py</code>
The student simulator	<code>tutor_rl/student_env.py</code>
Tutors (baselines + RL policy)	<code>tutor_rl/tutors.py</code>
Validate the sim's "physics"	<code>tutor_rl/test_physics.py</code>
Train the RL tutor	<code>tutor_rl/train.py</code>
Evaluate vs baselines	<code>tutor_rl/evaluate.py</code>
Realistic LLM student (demos)	<code>tutor_rl/llm_student.py</code>
Distil RL → training data	<code>tutor_rl/gen_sft_data.py</code>
Calibration flywheel	<code>tutor_rl/calibration/</code> + <code>tutor_rl/calibrate.py</code>
Engine README	<code>tutor_rl/README.md</code>

8. How to operate it

Research engine (laptop, only needs `numpy`):

```
cd tutor_rl
python3 test_physics.py      # validate the simulator behaves sanely
python3 evaluate.py          # train the RL tutor + compare to baselines
```

Refresh calibration from real (de-identified) data:

```
bash tutor_rl/calibration/refresh.sh # pulls VM stats, recalibrates the simulator
```

Check the live WhatsApp templates (read-only):

```
ssh -i ~/.ssh/gurukul_key dk_trigun@20.219.2.53 'node /tmp/wa_setup_templates.mjs'
```

Golden rules when touching the live tutor: - Never restart **wa-bridge** while a student may be mid-conversation. - Skill (method) changes deploy by file copy — no restart, picked up next session. - See the **maintain-trigunai-system** skill before any change to a live property.

9. Status at a glance

Part	Status
Acharya live tutor (WhatsApp + web)	✅ Live, daily use
Proven teaching method in the prompt	✅ Strong; upgraded 2026-06-29 (elicit-first + misconception repair)
WhatsApp automated messaging (templates + SRS timer)	✅ Live (Marketing category — recategorize before scale)
RL research engine (sim + train + eval)	✅ Works end-to-end
Calibration flywheel (real → sim)	✅ Plumbed + privacy-safe; data still thin
RL tutor beating the hand-built tutor	⌚ Needs bigger policy (MLP+PPO) + more real data

10. Honest limits (don't oversell)

- The simulator runs on **learning-science priors, not yet fully calibrated** to real students. Tutor *rankings* are meaningful; absolute numbers aren't "real learning gains."
- The RL policy is small (linear) — enough to prove learning beats heuristics on the easy baselines, not enough to beat a strong hand-built tutor yet.
- **Most of the teaching gain is the *method*, not the learned sequencer** — which is why the free, today win was improving Acharya's prompt. The RL sequencer is the *additional* margin.
- The flywheel is real but **data-thin** until the cohort grows and the daily recalls accumulate answers.

11. Glossary

- **Acharya** — our live AI tutor persona (Sanskrit for "teacher/guide").
- **SRS (Spaced Repetition)** — reviewing a concept at expanding intervals so it sticks.
- **Mastery gate** — don't advance until the student proves they understood.
- **Misconception** — a *specific wrong mental model* (not just "didn't know").
- **Learner Profile / Learner Model** — the per-student record of what they know.
- **Reinforcement Learning (RL)** — learning by trial-and-error to maximize a reward.
- **Policy** — the trained decision-maker (here: the tutor choosing its next move).
- **Student simulator** — a fake-but-realistic student the tutor practises on.
- **Calibration** — tuning the simulator so it matches real students.

- **Flywheel** — the self-reinforcing loop: real data → better sim → better tutor → better teaching → more/better real data.
 - **WABA** — WhatsApp Business Account (owns the phone number + message templates).
 - **Bloom's 2-sigma** — the finding that 1-on-1 mastery tutoring $\approx$ +2 std-devs over lectures.
-

Companion docs: [TUTOR_RL_STUDENT_SIMULATOR_SPEC.md](#) (deep design of the simulator), [tutor_rl/README.md](#) (how to run the engine), and the [maintain-trigunai-system](#) skill (operating the live properties safely).